

Atividade de Programação Guiada 2 (PG2)

INF01151 - Sistemas Operacionais II - Prof. Eder John Scheid

8 de outubro de 2025

Este é o **quarto** de quatro exercícios práticos que compõem parte da avaliação prática (MPP). Os quatro exercícios, no total, valem 1,0 ponto da MPP; portanto, cada exercício corresponde a 0,25 ponto (25% do total). A PG1 representa o terceiro exercício.

Objetivo

Praticar a criação de um Web Service básico em Python simulando uma calculadora que recebe requisições GET e POST utilizando a biblioteca Flask.

Introdução ao Flask

O **Flask** é um *microframework* em Python voltado para o desenvolvimento de aplicações Web. É chamado de “micro” porque fornece apenas o essencial para criar um servidor HTTP e lidar com rotas, sem impor estruturas complexas ou dependências pesadas. Isso o torna leve, flexível e ideal tanto para prototipagem rápida quanto para criação de *APIs REST* e *microserviços*. Documentação: <https://flask.palletsprojects.com/en/stable/>

Instalação

Para instalar o Flask (considerando Ubuntu 20.04), execute:

```
pip3 install flask
```

ou

```
sudo apt install python3-flask
```

Algumas características do Flask são:

- **Roteamento simples:** define *endpoints* com decoradores (`@app.route()`, `@app.get()`, `@app.post()`).
- **Servidor embutido:** executa localmente com `flask run` ou `app.run()`, ideal para testes.
- **Respostas JSON integradas:** suporte nativo via `jsonify()` para criar *APIs REST*.

- **Extensível:** permite adicionar módulos (autenticação, templates, dentre outros).
- **Stateless:** cada requisição é independente, seguindo o princípio REST. Ver o slide 19 da Aula 16 da disciplina.

Atividade Guiada

1. Faça o download da implementação base: baixe da página do moodle o arquivo PG2-WebServices.zip, e descompacte-o. Verifique no diretório descompactado quais arquivos estão presentes e examine o conteúdo deles. Os arquivos fonte especificam um cliente realizando requisições (`client.py`) e o servidor implementando Web Service (`app.py`).
2. Execute o arquivo `app.py`, abra outro terminal e execute o arquivo `client.py`.
3. Você deve ter encontrado erros, certo?
4. Agora vamos implementar as rotas e funções para somar e multiplicar, um utilizando GET e outro POST. Todos os códigos abaixo devem ser realizados apenas no arquivo `app.py`.

Implementação da rota GET /calculadora/somar/{a}/{b}

1. Defina o *decorator* da rota (método GET)

- Escreva exatamente:

```
@app.get("/calculadora/somar/<int:a>/<int:b>")
```

- Isso vincula a URL /calculadora/somar/<a>/ (com a e b inteiros) à função Python que virá **imediatamente abaixo**.

2. Defina a função somar imediatamente abaixo do decorator

- A função deve vir **logo na linha seguinte** ao decorator, com a mesma indentação do nível de módulo:

```
def somar(a, b):
```

- Não insira linhas em branco entre o decorator e a função; isso mantém a associação explícita da rota à função.

3. Calcule o resultado dentro da função

- Some os operandos:

```
soma = a + b
```

4. Monte o dicionário de resposta no contrato combinado

- Utilize chaves claras e consistentes:

```
retorno = {
    "operacao": "soma",
    "a": a,
    "b": b,
    "resultado": soma
}
```

5. Retorne JSON + status HTTP

- Use `jsonify` para serializar e retorne 200 OK:

```
return jsonify(retorno), 200
```

- Não use códigos 201 (Created) para GET; 200 é o correto para leitura bem-sucedida.

6. Teste

- Suba o servidor: `python3 app.py`
- Abra um outro terminal e execute `python3 client.py`:
- Saída esperada (trecho):

```
Echo GET: {'echo': 'hello'}
Echo POST: {'echo': 'Hello, UFRGS'}
GET soma 2+3: {'a': 2, 'b': 3, 'operacao': 'soma', 'resultado': 5}
Traceback (most recent call last):
...
```

Implementação da rota POST /calculadora/multiplicar

1. Defina o *decorator* da rota (método POST)

- Escreva exatamente:

```
@app.post("/calculadora/multiplicar")
```

- Este decorator deve ficar **imediatamente acima** da função que o implementa, sem linhas em branco no meio.

2. Defina a função multiplicar imediatamente abaixo do decorator

- Na linha seguinte ao decorator:

```
def multiplicar():
```

- Não quebre a associação decorator → função com linhas soltas.

3. Leia e valide o corpo JSON

- Obtenha o payload com tolerância a ausência:

```
data = request.get_json(silent=True) or {}
```

- Leia os campos esperados:

```
a = data.get("a")
b = data.get("b")
```

- Se algum campo estiver ausente, retorne 400 Bad Request:

```
if a is None:
    abort(400, description="JSON invalido: esperado {'a': int}")
if b is None:
    abort(400, description="JSON invalido: esperado {'b': int}")
```

- (*Recomendado*) Converta para int e trate tipo inválido para evitar *concatenação* de strings:

```
try:
    a = int(a)
    b = int(b)
except (TypeError, ValueError):
    abort(400, description="'a' e 'b' devem ser inteiros")
```

4. Calcule o resultado

- Faça a multiplicação:

```
multiplicacao = a * b
```

5. Monte o dicionário de resposta no contrato combinado

- Utilize chaves claras e consistentes:

```
retorno = {
    "operacao": "multiplicar",
    "a": a,
    "b": b,
    "resultado": multiplicacao,
}
```

6. Retorne JSON + status HTTP

- Para POST bem-sucedido, use 201 Created:

```
return jsonify(retorno), 201
```

7. Teste

- Suba o servidor: `python3 app.py`
- Abra um outro terminal e execute `python3 client.py`:
- Saída esperada (trecho):

```
Echo GET: {'echo': 'hello'}
Echo POST: {'echo': 'Hello, UFRGS'}
GET soma 2+3: {'a': 2, 'b': 3, 'operacao': 'soma', 'resultado': 5}
POST multiplicar 4*5: {'a': 4, 'b': 5,
'operacao': 'multiplicar', 'resultado': 20}
Traceback (most recent call last):
...
```

Notas

O parâmetro `silent=True` em `request.get_json()` evita que o Flask gere uma exceção automática (`BadRequest`) ao receber um corpo de requisição inválido. Com esse parâmetro, a função retorna `None` em vez de encerrar o processamento, permitindo que o desenvolvedor trate manualmente o erro e retorne uma resposta mais adequada ao cliente.

Os códigos de status HTTP indicam o resultado da operação executada pelo servidor, conforme a *RFC 7231*¹. O código **200 (OK)** é utilizado quando a requisição foi processada com sucesso e o recurso foi retornado no corpo da resposta. O código **201 (Created)** indica que uma nova entidade foi criada com sucesso (comum em operações `POST`). Já o código **400 (Bad Request)** deve ser usado quando o cliente envia uma requisição inválida, como um JSON malformatado ou dados incorretos (exemplo: tentativa de dividir por zero).

Exercício

Seguindo os exemplos dos endpoints `GET somar` e `POST multiplicar`, implemente os demais endpoints necessários para que o programa `client.py` seja executado sem erros. Para isso, desenvolva as funções **GET** `/calculadora/subtrair` e **POST** `/calculadora/dividir`. Lembre-se de realizar o tratamento de erro para o caso de divisão por zero, retornando o código HTTP adequado.

Entrega

Se você chegou até aqui, parabéns! Faça o envio do relatório do programa no Moodle, e seu objetivo na atividade de programação guiada estará cumprido. O relatório deverá incluir o código fonte desenvolvido e screenshots do funcionamento do serviço e do cliente interagindo com o Web Service.

¹<https://datatracker.ietf.org/doc/html/rfc7231>